

■ Golden Implementation Prompt

Sprint 2: User Authentication

Authentic AI Voice MVP • Register → Login → Protected Routes • JWT Auth

Sprint 2

✓ Frontend: Dev 2

Stack: Next.js 14 + FastAPI + JWT

1. Role & Context

You are an expert Frontend Developer (Next.js 14 + TypeScript). You are implementing **Sprint 2** of the "Authentic AI Voice" web application — an AI-powered writing tool that learns a user's writing style.

This sprint adds real user authentication on top of the working Sprint 1 core loop. The backend JWT endpoints (**POST /api/auth/register**, **POST /api/auth/login**, **GET /api/auth/me**) are already implemented by Dev 1. Your job is entirely frontend: build the auth UI, store the JWT token, protect all routes, and wire the logout flow.

Dev 2 Scope (Frontend Only): Login page, Register page, Next.js middleware for route protection, dashboard logout button + user avatar, update api.ts 401 interceptor to clear cookie. You do NOT modify any backend files.

2. Tech Stack (Strictly Follow This)

Frontend Framework	Next.js 14.2.35 App Router
Language	
State Management	Zustand 5.0.13
HTTP Client	
Styling	Tailwind CSS 3.4.19 + CSS variables in globals.css
Auth Mechanism	
Route Protection	Next.js middleware.ts at frontend root
No New Packages	

i 3. Project Structure — Files to Touch

```

frontend/
  ■■■ middleware.ts ← CREATE THIS (route protection)
  ■■■ lib/
  ■ ■■■ api.ts ← MODIFY: 401 interceptor clears cookie
  ■■■ app/
  ■■■ login/page.tsx ← REPLACE placeholder with real form
  ■■■ register/page.tsx ← REPLACE placeholder with real form
  ■■■ dashboard/page.tsx ← ADD: logout button + user email avatar

DO NOT TOUCH (Sprint 1 – leave as-is):
  ■■■ components/editor/* ← Sprint 1 complete
  ■■■ hooks/useAutoSave.ts ← Sprint 1 complete
  ■■■ store/editorStore.ts ← Sprint 1 complete
  ■■■ app/documents/[id]/page.tsx ← Sprint 1 complete

```

■ 4. Token Storage Strategy (Critical)

Next.js middleware runs on the **server edge** and cannot read **localStorage**. Solution: store the token in **both** localStorage (for Axios interceptor) AND a cookie (for middleware). Both must always be kept in sync.

On login / register success:

```

localStorage.setItem('auth_token', data.access_token);
document.cookie = `auth_token=${data.access_token}; path=/; max-age=${60*60*24};
SameSite=Strict`;

```

On logout or 401 response:

```

localStorage.removeItem('auth_token');
document.cookie = 'auth_token=; path=/; max-age=0';

```

á 5. Frontend Implementation Tasks

S2-F1

REPLAC
E

app/login/page.tsx

- Full-screen centered layout — dark theme matching Sprint 1 globals.css
- AppLogo component at top (import from @/components/ui/AppLogo)
- Email input + Password input with show/hide toggle
- 'Sign In' button using .btn-primary CSS class
- Inline error display (not toast) on 401: 'Invalid email or password'
- Loading spinner inside button during API call
- On success: store token in localStorage + cookie, router.push('/dashboard')
- Link to /register at bottom

S2-F2**REPLAC
E***app/register/page.tsx*

- Same dark card layout as login — consistent design
- Email, Password, Confirm Password inputs
- Client-side: password $\geq$ 8 chars validation before API call
- Client-side: passwords match validation before API call
- On 409 from API: 'An account with this email already exists.'
- On success: store token in localStorage + cookie, router.push('/dashboard')
- Link to /login at bottom

S2-F3**CREATE***middleware.ts (root)*

- Protect all routes except /login, /register, /_next/*, /static/*, /favicon.ico
- Read auth_token from cookies (NOT localStorage — server-side only)
- No token found → redirect to /login?from=
- Use NextResponse.redirect() with correct URL construction
- Export config.matcher to scope middleware correctly

S2-F4**MODIFY***lib/api.ts + app/dashboard/page.tsx*

- api.ts: update 401 interceptor to also clear cookie (document.cookie line)
- dashboard/page.tsx: add useEffect to call GET /api/auth/me on mount
- Show email first letter as avatar initial (replace '?' placeholder)
- Add 'Sign out' button next to avatar using handleLogout()
- handleLogout: clear localStorage + cookie, router.push('/login')

■ 6. API Reference — Sprint 2 Auth Endpoints

Method	Path	Body	Response
POST	<i>/api/auth/register</i>	<i>{email, password}</i>	<i>{access_token, token_type, user}</i>
	<i>/api/auth/login</i>	<i>{email, password}</i>	<i>{access_token, token_type, user}</i>
GET	<i>/api/auth/me</i>	— (Bearer header)	<i>{id, email, created_at}</i>

7. Definition of Done — Sprint 2 Frontend

Area	Criterion	Done
Login Page	/login renders with dark theme, AppLogo, email + password inputs	■

Login Page	Valid credentials → token stored in localStorage AND cookie → redirect /dashboard	■
Login Page		■
Login Page	Button shows loading spinner during request	■
Login Page		■
Register Page	/register renders with consistent dark styling	■
Register Page		■
Register Page	Passwords don't match → client-side error, no API call made	■
Register Page		■
Register Page	Duplicate email (409) → 'An account with this email already exists.'	■
Middleware		■
Middleware	Visit /documents/[id] with no token → redirects to /login	■
Middleware		■
Middleware	/_next/* and /static/* paths are NOT blocked	■
Dashboard		■
Dashboard	'Sign out' button appears next to avatar	■
Dashboard		■
api.ts	401 response handler clears BOTH localStorage AND cookie	■
TypeScript		■
E2E	Register → dashboard → create doc → sign out → login → doc still there	■

- 8. Critical Implementation Notes

1. "use client" directive	Required on login/page.tsx and register/page.tsx — they use useState/useEffect. middleware.ts does NOT get this directive.
2. CSS variables only	Use var(--color-*) and var(--radius-*) for ALL colors and radii. Never hardcode hex values. See globals.css for the full token list.
3. Existing CSS classes	Use .btn-primary for the main CTA button. Use .btn-ghost for the logout button. Never recreate these with inline styles.

4. Token key is exactly 'auth_token'	The Axios interceptor in api.ts reads localStorage.getItem('auth_token'). The middleware reads cookies.get('auth_token'). Use the exact string — not 'token' or 'jwt'.
5. No new dependencies	Do NOT run npm install for any new package. Everything needed (Axios, Zustand, Next.js router) is already installed from Sprint 1.
6. AppLogo import	Use: import AppLogo from '@components/ui/AppLogo' — default export for the full logo. Use import { Applcon } from '@components/ui/AppLogo' for the icon-only variant.
7. Error extraction pattern	Always extract: err?.response?.data?.detail 'Fallback message'. Never show raw error objects or err.message from Axios.
8. TypeScript strict mode	Run npx tsc --noEmit before marking any task done. The project has strict: true in tsconfig.json. React.FormEvent, not just FormEvent.